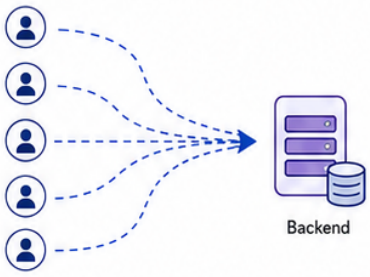
Day 32: Request Coalescing, Cache Stampede Protection, and One-Flight

Preventing cache-miss storms by collapsing concurrent identical requests into one backend fetch.

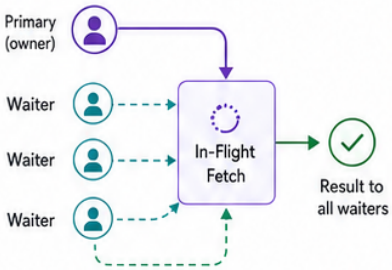
1 Why Coalescing Matters

- Hot keys can trigger many concurrent misses after TTL expiry.
- Without protection, all misses hit the backend at once.
- Coalescing keeps only one in-flight fetch per key.



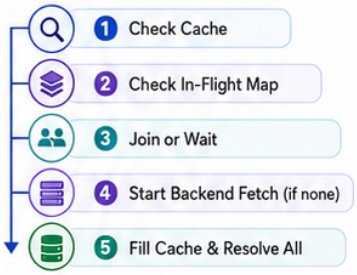
2 One-Flight Model

- First miss becomes the primary fetch owner.
- Later misses join as waiters.
- When fetch completes, all waiters receive the same value.



3 Stampede Protection Flow

- Check cache first.
- On miss, check in-flight map.
- Join waiters if a fetch already exists.
- Otherwise start one backend fetch.
- Completion fills cache and resolves all waiters.

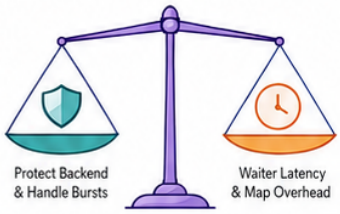


4 TTL and Freshness

- Longer TTL lowers backend load but increases staleness.
- Short TTL improves freshness but raises miss frequency.
- Coalescing reduces miss amplification even with short TTL.

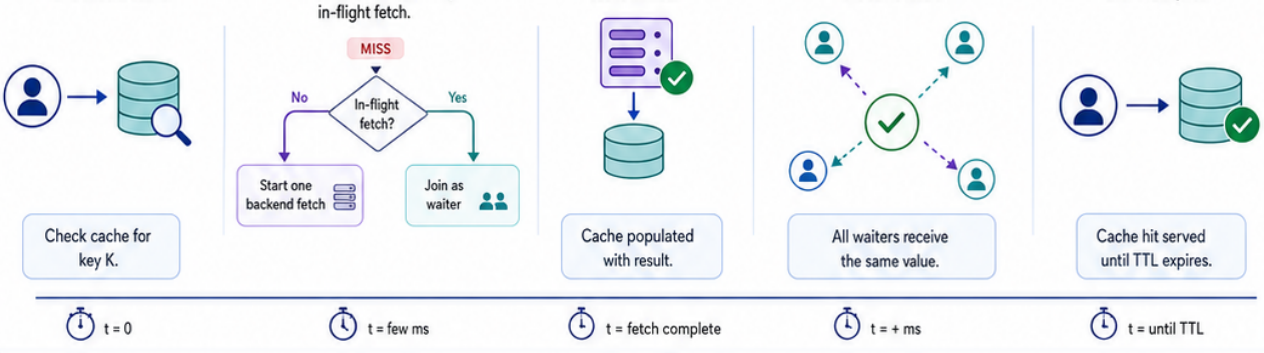
5 Trade-Offs

- Coalescing protects the backend and improves burst handling.
- Waiters inherit the primary fetch latency.
- Per-key in-flight maps need cleanup and bounded memory.



Sample Request Coalescing Flow

1. Request arrives and checks cache.
2. Cache miss starts one fetch or joins existing in-flight fetch.
3. Backend completes, cache is filled.
4. Waiting requests share the same result.
5. Later requests hit cache until TTL expires.



One backend fetch can serve many concurrent misses, reducing duplicate work and protecting the backend from cache stampedes.

System Design - Day 32

Request Coalescing, Cache Stampede Protection, and One-Flight

Prepared from Day-32 notes and sample implementation

Overview from README.md

Day 32: Request Coalescing, Cache Stampede Protection, and One-Flight

Day 32 focuses on preventing cache-miss storms by collapsing concurrent identical requests into one backend fetch.

It covers:
• request coalescing fundamentals
• one-flight execution model
• cache stampede protection behavior
• waiter fan-in and shared completion
• cache TTL impact on backend load
• latency and capacity trade-offs

In short, Day 32 is about ensuring one backend call serves many concurrent misses for the same key instead of triggering duplicate work.

Core ideas

Why coalescing matters

• Hot keys can trigger many concurrent misses after TTL expiry.
• Without protection, all misses hit the backend at once.
• Coalescing keeps only one in-flight fetch per key and lets others wait.

One-flight model

• First miss becomes the primary fetch owner.
• Later misses for the same key join as waiters.
• When fetch completes, all waiters receive the same value.

Stampede protection flow

• Request checks cache first.
• On miss, request checks in-flight map for the key.
• If in-flight exists, join waiters; otherwise create new backend fetch.
• Completion fills cache and resolves all waiters.

TTL and freshness

• Longer TTL lowers backend load but increases staleness window.
• Short TTL improves freshness but raises miss frequency.
• Coalescing reduces miss amplification even when TTL is short.

Trade-offs

• Coalescing improves backend protection and often p95 under bursts.
• Waiters inherit primary fetch latency and may block longer than cache hits.

• Per-key lock map requires cleanup and bounded memory.

Day-32 sample: Request Coalescing Simulator

This repository includes a request coalescing sample in both Python and Java.

Functional requirements

• Accept requests arriving at specific timestamps for one hot key
• Apply cache TTL checks before backend fetch
• Allow only one in-flight backend fetch per key
• Attach concurrent misses as waiters to the same in-flight fetch
• Return request-level latencies and aggregate backend/load metrics

High-level components

• `CacheEntry`: cached value and expiry time
• `InFlightFetch`: primary owner, waiters, and backend finish time
• `RequestCoalescer`: orchestration for cache checks and one-flight logic
• `SimulationResult`: request outcomes, timeline, and metrics

Data flow

1. Request arrives and checks cache validity
2. Cache miss either starts new fetch or joins existing in-flight fetch
3. Backend completion writes cache entry and resolves all waiting requests
4. Later requests hit cache until TTL expires
5. Simulator reports hit ratios, coalesced requests, and backend calls saved

Project structure

Day-32/
README.md
request-coalescing-sample/
python/
request_coalescing.py
demo.py
java/
RequestCoalescingSystem.java
Main.java

Run the sample

Python

```
cd request-coalescing-sample\python
python demo.py
```

Java

```
cd request-coalescing-sample\java
javac Main.java RequestCoalescingSystem.java
java Main
```

Interview takeaways

- One-flight is a practical defense against cache stampedes
- Coalesced waiters can dramatically reduce duplicate backend load
- Cache TTL and coalescing should be tuned together
- Hot-key observability is required for safe rollout
- Coalescing complements stale-while-revalidate and rate limiting

Next improvements

- Extend to multi-key simulation with per-key in-flight map
- Add stale-while-revalidate mode for instant stale responses
- Add jittered TTL expiry to reduce synchronized misses
- Add timeout/cancellation for waiters
- Add per-key memory guardrails for waiter lists

Program Listings

1. request-coalescing-sample\python\request_coalescing.py

```
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum


class ResolutionPath(Enum):
    CACHE_HIT = "CACHE_HIT"
    PRIMARY_FETCH = "PRIMARY_FETCH"
    COALESCED_WAIT = "COALESCED_WAIT"


@dataclass(frozen=True)
class RequestOutcome:
    request_id: int
    arrival_ms: int
    completion_ms: int
    latency_ms: int
    path: ResolutionPath


@dataclass(frozen=True)
class SimulationResult:
    outcomes: list[RequestOutcome]
    timeline: list[str]
    total_requests: int
    cache_hits: int
    coalesced_requests: int
    backend_calls: int
    backend_calls_saved: int
    average_latency_ms: float


@dataclass
class CacheEntry:
    value: str
    expires_at_ms: int


@dataclass
class InFlightFetch:
    value: str
    started_at_ms: int
    finishes_at_ms: int
    primary_request_id: int
    waiter_request_ids: list[int]


class RequestCoalescer:
    def simulate(
        self,
        request_arrivals_ms: list[int],
        backend_latencies_ms: list[int],
        ttl_ms: int,
    ) -> SimulationResult:
        if not request_arrivals_ms:
            raise ValueError("request_arrivals_ms must be non-empty")
        if any(request_arrivals_ms[index] < request_arrivals_ms[index - 1] for index in range(1, len(request_arriv
            raise ValueError("request_arrivals_ms must be non-decreasing")
        if any(latency <= 0 for latency in backend_latencies_ms):
            raise ValueError("backend_latencies_ms must contain positive values")
        if ttl_ms <= 0:
            raise ValueError("ttl_ms must be positive")

        timeline: list[str] = [f"simulation started ttl={ttl_ms}ms"]
        outcomes: dict[int, RequestOutcome] = {}

        cache: CacheEntry | None = None
        inflight: InFlightFetch | None = None
        backend_call_count = 0
        backend_latency_index = 0
        def complete_inflight_if_ready(current_time_ms: int) -> None:
```

```
        nonlocal inflight, cache
        if inflight is None or inflight.finishes_at_ms > current_time_ms:
            return

        finish = inflight.finishes_at_ms
        cache = CacheEntry(value=inflight.value, expires_at_ms=finish + ttl_ms)
        timeline.append(
            f"inflight completed at t={finish}ms cache_expires_at={cache.expires_at_ms}ms"
        )

        primary_id = inflight.primary_request_id
        primary_arrival = request_arrivals_ms[primary_id]
        outcomes[primary_id] = RequestOutcome(
            request_id=primary_id,
            arrival_ms=primary_arrival,
            completion_ms=finish,
            latency_ms=finish - primary_arrival,
            path=ResolutionPath.PRIMARY_FETCH,
        )

        for request_id in inflight.waiter_request_ids:
            arrival = request_arrivals_ms[request_id]
            outcomes[request_id] = RequestOutcome(
                request_id=request_id,
                arrival_ms=arrival,
                completion_ms=finish,
                latency_ms=finish - arrival,
                path=ResolutionPath.COALESCED_WAIT,
            )

        inflight = None

    for request_id, arrival_ms in enumerate(request_arrivals_ms):
        complete_inflight_if_ready(arrival_ms)

        if cache is not None and arrival_ms < cache.expires_at_ms:
            timeline.append(f"r{request_id} t={arrival_ms}ms cache_hit")
            outcomes[request_id] = RequestOutcome(
                request_id=request_id,
                arrival_ms=arrival_ms,
                completion_ms=arrival_ms,
                latency_ms=0,
                path=ResolutionPath.CACHE_HIT,
            )
            continue

        if inflight is not None and arrival_ms < inflight.finishes_at_ms:
            timeline.append(
                f"r{request_id} t={arrival_ms}ms joined_inflight "
                f"(finishes_at={inflight.finishes_at_ms}ms)"
            )
            inflight.waiter_request_ids.append(request_id)
            continue

        latency = backend_latencies_ms[min(backend_latency_index, len(backend_latencies_ms) - 1)]
        backend_latency_index += 1
        backend_call_count += 1
        finish_ms = arrival_ms + latency
        inflight = InFlightFetch(
            value=f"value_v{backend_call_count}",
            started_at_ms=arrival_ms,
            finishes_at_ms=finish_ms,
            primary_request_id=request_id,
            waiter_request_ids=[],
        )
        timeline.append(
            f"r{request_id} t={arrival_ms}ms started_backend_fetch "
            f"(latency={latency}ms finish={finish_ms}ms)"
        )

    complete_inflight_if_ready(10**9)

    ordered_outcomes = [outcomes[index] for index in sorted(outcomes.keys())]
    if len(ordered_outcomes) != len(request_arrivals_ms):
        raise RuntimeError("internal error: missing request outcomes")

    cache_hits = sum(1 for outcome in ordered_outcomes if outcome.path == ResolutionPath.CACHE_HIT)
```

```
        coalesced = sum(1 for outcome in ordered_outcomes if outcome.path == ResolutionPath.COALESCED_WAIT)
        avg_latency = sum(outcome.latency_ms for outcome in ordered_outcomes) / len(ordered_outcomes)
        backend_calls_saved = coalesced

        timeline.append(
            f"finished total={len(ordered_outcomes)} cache_hits={cache_hits} "
            f"coalesced={coalesced} backend_calls={backend_call_count} "
            f"saved_calls={backend_calls_saved}"
        )
        return SimulationResult(
            outcomes=ordered_outcomes,
            timeline=timeline,
            total_requests=len(ordered_outcomes),
            cache_hits=cache_hits,
            coalesced_requests=coalesced,
            backend_calls=backend_call_count,
            backend_calls_saved=backend_calls_saved,
            average_latency_ms=avg_latency,
        )
```

2. request-coalescing-sample\python\demo.py

```
from request_coalescing import RequestCoalescer


def print_result(title: str, arrivals: list[int], latencies: list[int], ttl_ms: int) -> None:
    coalescer = RequestCoalescer()
    result = coalescer.simulate(arrivals, latencies, ttl_ms)

    print(title)
    print(
        f"  total={result.total_requests} cache_hits={result.cache_hits} "
        f"coalesced={result.coalesced_requests} backend_calls={result.backend_calls} "
        f"saved_calls={result.backend_calls_saved} avg_latency={result.average_latency_ms:.1f}ms"
    )
    for outcome in result.outcomes:
        print(
            f"  r{outcome.request_id}: arrival={outcome.arrival_ms}ms "
            f"completion={outcome.completion_ms}ms latency={outcome.latency_ms}ms "
            f"path={outcome.path.value}"
        )
    for line in result.timeline:
        print(" ", line)


def main() -> None:
    print_result(
        "burst after cache expiry (stampede protected by one-flight)",
        arrivals=[0, 2, 4, 6, 40],
        latencies=[25, 30],
        ttl_ms=30,
    )

    print()
    print_result(
        "spaced traffic (little coalescing opportunity)",
        arrivals=[0, 35, 70],
        latencies=[20, 20, 20],
        ttl_ms=10,
    )


if __name__ == "__main__":
    main()
```

3. request-coalescing-sample\java\RequestCoalescingSystem.java

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class RequestCoalescingSystem {
    public enum ResolutionPath {
        CACHE_HIT,
        PRIMARY_FETCH,
```

```
    COALESCED_WAIT
}

public static final class RequestOutcome {
    private final int requestId;
    private final int arrivalMs;
    private final int completionMs;
    private final int latencyMs;
    private final ResolutionPath path;

    public RequestOutcome(int requestId, int arrivalMs, int completionMs, int latencyMs, ResolutionPath path)
        this.requestId = requestId;
        this.arrivalMs = arrivalMs;
        this.completionMs = completionMs;
        this.latencyMs = latencyMs;
        this.path = path;
    }

    public int getRequestId() {
        return requestId;
    }

    public int getArrivalMs() {
        return arrivalMs;
    }

    public int getCompletionMs() {
        return completionMs;
    }

    public int getLatencyMs() {
        return latencyMs;
    }

    public ResolutionPath getPath() {
        return path;
    }
}

public static final class SimulationResult {
    private final List<RequestOutcome> outcomes;
    private final List<String> timeline;
    private final int totalRequests;
    private final int cacheHits;
    private final int coalescedRequests;
    private final int backendCalls;
    private final int backendCallsSaved;
    private final double averageLatencyMs;

    public SimulationResult(
        List<RequestOutcome> outcomes,
        List<String> timeline,
        int totalRequests,
        int cacheHits,
        int coalescedRequests,
        int backendCalls,
        int backendCallsSaved,
        double averageLatencyMs
    ) {
        this.outcomes = outcomes;
        this.timeline = timeline;
        this.totalRequests = totalRequests;
        this.cacheHits = cacheHits;
        this.coalescedRequests = coalescedRequests;
        this.backendCalls = backendCalls;
        this.backendCallsSaved = backendCallsSaved;
        this.averageLatencyMs = averageLatencyMs;
    }

    public List<RequestOutcome> getOutcomes() {
        return outcomes;
    }

    public List<String> getTimeline() {
        return timeline;
    }
    public int getTotalRequests() {
```

```
        return totalRequests;
    }

    public int getCacheHits() {
        return cacheHits;
    }

    public int getCoalescedRequests() {
        return coalescedRequests;
    }

    public int getBackendCalls() {
        return backendCalls;
    }

    public int getBackendCallsSaved() {
        return backendCallsSaved;
    }

    public double getAverageLatencyMs() {
        return averageLatencyMs;
    }
}

private static final class CacheEntry {
    private final String value;
    private final int expiresAtMs;

    private CacheEntry(String value, int expiresAtMs) {
        this.value = value;
        this.expiresAtMs = expiresAtMs;
    }
}

private static final class InFlightFetch {
    private final String value;
    private final int startedAtMs;
    private final int finishesAtMs;
    private final int primaryRequestId;
    private final List<Integer> waiterRequestIds;

    private InFlightFetch(
        String value,
        int startedAtMs,
        int finishesAtMs,
        int primaryRequestId,
        List<Integer> waiterRequestIds
    ) {
        this.value = value;
        this.startedAtMs = startedAtMs;
        this.finishesAtMs = finishesAtMs;
        this.primaryRequestId = primaryRequestId;
        this.waiterRequestIds = waiterRequestIds;
    }
}

public static final class RequestCoalescer {
    public SimulationResult simulate(List<Integer> requestArrivalsMs, List<Integer> backendLatenciesMs, int tt
        if (requestArrivalsMs == null || requestArrivalsMs.isEmpty()) {
            throw new IllegalArgumentException("requestArrivalsMs must be non-empty");
        }
        for (int i = 1; i < requestArrivalsMs.size(); i++) {
            if (requestArrivalsMs.get(i) < requestArrivalsMs.get(i - 1)) {
                throw new IllegalArgumentException("requestArrivalsMs must be non-decreasing");
            }
        }
        if (backendLatenciesMs == null || backendLatenciesMs.isEmpty()) {
            throw new IllegalArgumentException("backendLatenciesMs must be non-empty");
        }
        for (int latency : backendLatenciesMs) {
            if (latency <= 0) {
                throw new IllegalArgumentException("backendLatenciesMs must contain positive values");
            }
        }
        if (ttlMs <= 0) {
            throw new IllegalArgumentException("ttlMs must be positive");
        }
```

```
List<String> timeline = new ArrayList<>();
timeline.add("simulation started ttl=" + ttlMs + "ms");
Map<Integer, RequestOutcome> outcomes = new HashMap<>();

Holder holder = new Holder();
holder.cache = null;
holder.inflight = null;
int backendCallCount = 0;
int backendLatencyIndex = 0;

for (int requestId = 0; requestId < requestArrivalsMs.size(); requestId++) {
    int arrivalMs = requestArrivalsMs.get(requestId);
    completeInFlightIfReady(arrivalMs, ttlMs, requestArrivalsMs, outcomes, timeline, holder);

    if (holder.cache != null && arrivalMs < holder.cache.expiresAtMs) {
        timeline.add("r" + requestId + " t=" + arrivalMs + "ms cache_hit");
        outcomes.put(
            requestId,
            new RequestOutcome(requestId, arrivalMs, arrivalMs, 0, ResolutionPath.CACHE_HIT)
        );
        continue;
    }

    if (holder.inflight != null && arrivalMs < holder.inflight.finishesAtMs) {
        timeline.add(
            "r"
                + requestId
                + " t="
                + arrivalMs
                + "ms joined_inflight (finishes_at="
                + holder.inflight.finishesAtMs
                + "ms)"
        );
        holder.inflight.waiterRequestIds.add(requestId);
        continue;
    }

    int latency = backendLatenciesMs.get(Math.min(backendLatencyIndex, backendLatenciesMs.size() - 1))
    backendLatencyIndex += 1;
    backendCallCount += 1;
    int finishMs = arrivalMs + latency;
    holder.inflight = new InFlightFetch(
        "value_v" + backendCallCount,
        arrivalMs,
        finishMs,
        requestId,
        new ArrayList<>()
    );
    timeline.add(
        "r"
            + requestId
            + " t="
            + arrivalMs
            + "ms started_backend_fetch (latency="
            + latency
            + "ms finish="
            + finishMs
            + "ms)"
    );
}

completeInFlightIfReady(1_000_000_000, ttlMs, requestArrivalsMs, outcomes, timeline, holder);

List<RequestOutcome> ordered = new ArrayList<>();
for (int i = 0; i < requestArrivalsMs.size(); i++) {
    RequestOutcome outcome = outcomes.get(i);
    if (outcome == null) {
        throw new IllegalStateException("internal error: missing request outcomes");
    }
    ordered.add(outcome);
}

int cacheHits = 0;
int coalesced = 0;
int totalLatency = 0;
for (RequestOutcome outcome : ordered) {
    totalLatency += outcome.getLatencyMs();
```

```
        if (outcome.getPath() == ResolutionPath.CACHE_HIT) {
            cacheHits += 1;
        } else if (outcome.getPath() == ResolutionPath.COALESCED_WAIT) {
            coalesced += 1;
        }
    }

    int savedCalls = coalesced;
    double avgLatency = (double) totalLatency / ordered.size();
    timeline.add(
        "finished total="
            + ordered.size()
            + " cache_hits="
            + cacheHits
            + " coalesced="
            + coalesced
            + " backend_calls="
            + backendCallCount
            + " saved_calls="
            + savedCalls
    );

    return new SimulationResult(
        ordered,
        timeline,
        ordered.size(),
        cacheHits,
        coalesced,
        backendCallCount,
        savedCalls,
        avgLatency
    );
}

private static void completeInFlightIfReady(
    int currentTimeMs,
    int ttlMs,
    List<Integer> requestArrivalsMs,
    Map<Integer, RequestOutcome> outcomes,
    List<String> timeline,
    Holder holder
) {
    if (holder.inflight == null || holder.inflight.finishesAtMs > currentTimeMs) {
        return;
    }

    int finish = holder.inflight.finishesAtMs;
    holder.cache = new CacheEntry(holder.inflight.value, finish + ttlMs);
    timeline.add(
        "inflight completed at t="
            + finish
            + "ms cache_expires_at="
            + holder.cache.expiresAtMs
            + "ms"
    );

    int primaryId = holder.inflight.primaryRequestId;
    int primaryArrival = requestArrivalsMs.get(primaryId);
    outcomes.put(
        primaryId,
        new RequestOutcome(
            primaryId,
            primaryArrival,
            finish,
            finish - primaryArrival,
            ResolutionPath.PRIMARY_FETCH
        )
    );

    for (int requestId : holder.inflight.waiterRequestIds) {
        int arrival = requestArrivalsMs.get(requestId);
        outcomes.put(
            requestId,
            new RequestOutcome(
                requestId,
                arrival,
                finish,
```

```
                        finish - arrival,
                        ResolutionPath.COALESCED_WAIT
                    )
                );
            }

            holder.inflight = null;
        }
    }

    private static final class Holder {
        private CacheEntry cache;
        private InFlightFetch inflight;
    }
}
```

4. request-coalescing-sample\java\Main.java

```
import java.util.List;

public class Main {
    private static void printResult(String title, List<Integer> arrivals, List<Integer> latencies, int ttlMs) {
        RequestCoalescingSystem.RequestCoalescer coalescer = new RequestCoalescingSystem.RequestCoalescer();
        RequestCoalescingSystem.SimulationResult result = coalescer.simulate(arrivals, latencies, ttlMs);

        System.out.println(title);
        System.out.println(
            "  total="
                + result.getTotalRequests()
                + " cache_hits="
                + result.getCacheHits()
                + " coalesced="
                + result.getCoalescedRequests()
                + " backend_calls="
                + result.getBackendCalls()
                + " saved_calls="
                + result.getBackendCallsSaved()
                + " avg_latency="
                + String.format("%.1f", result.getAverageLatencyMs())
                + "ms"
        );

        for (RequestCoalescingSystem.RequestOutcome outcome : result.getOutcomes()) {
            System.out.println(
                "  r"
                    + outcome.getRequestId()
                    + ": arrival="
                    + outcome.getArrivalMs()
                    + "ms completion="
                    + outcome.getCompletionMs()
                    + "ms latency="
                    + outcome.getLatencyMs()
                    + "ms path="
                    + outcome.getPath()
            );
        }

        for (String line : result.getTimeline()) {
            System.out.println("  " + line);
        }
    }

    public static void main(String[] args) {
        printResult(
            "burst after cache expiry (stampede protected by one-flight)",
            List.of(0, 2, 4, 6, 40),
            List.of(25, 30),
            30
        );

        System.out.println();
        printResult(
            "spaced traffic (little coalescing opportunity)",
            List.of(0, 35, 70),
            List.of(20, 20, 20),
            10
```

```
        );
    }
}
```